Final Project Defense requirements

- It’s a team project that should highlight use of multiple patterns (at least 6) from the course.
- Project topic ideas are provided in below slides, but, not limited to come up with own project topic.
- It’s highly recommended to form a group of at least 2-3 students for each project.
- The core requirement is that each student individually implements two design patterns at least.
- Collectively, the group must integrate, not less than, six distinct design patterns into a single cohesive project.
- These patterns must function in a coordinated manner, though not necessarily all interacting simultaneously.
- Direct interaction between some patterns is permissible and encouraged where appropriate.
- The primary objective is that all six design patterns are fully implemented and demonstrably operational within the unified project framework.
- Project Report must contain at least of Introductory part, Body part, Conclusion and Further work along with , Java / Go Codes, Screenshots of the Application and UML diagrams.
- Group Project must be submitted as Report in PDF Format into Moodle and Source code in zipped file.
- Final Project is to be defended in front of the Senior Lecturer and/or Teachers.

Final Project Topics (1/3)

- E-Commerce Shopping Cart: Use Builder for customizable product configurations, Observer to update inventory/price changes in real time, and Strategy for discount algorithms.
- Restaurant Ordering System: Factory (to create different meal objects), Decorator for extra toppings, Observer for order status notifications to kitchen and customer.
- Social Media Feed: Observer for live feed updates, Adapter to integrate third-party content APIs, Decorator to add filters (e.g. emoji overlay).
- Multimedia Editor: Command (from advanced patterns) and Strategy for various editing algorithms, Facade to simplify underlying audio/video libraries, Bridge for cross-platform UI (Windows/Mac).
- Smart Home Automation: Observer for event sensors (temperature, motion), Command/Strategy for device actions, Adapter for integrating different device protocols.
- Travel Itinerary Planner: Builder to assemble trip plan, Strategy for routing (fastest vs scenic), Facade to interface various travel APIs (hotels, flights).
- Medical Record System: Factory to create patient record types, Decorator to add patient alert flags (allergies, alerts), Observer for doctor notifications when records update.
- Game Development Engine: Bridge for graphics abstraction (2D/3D rendering engines), Strategy for AI behaviors, Factory for spawning game objects.
- Financial Trading Platform: Strategy for different trading algorithms, Observer for market data subscription, Facade for connecting to exchanges via a simplified interface.
- Document Processing Pipeline: Builder to construct document reports, Decorator to add metadata or watermarks, Adapter to handle multiple file formats (DOCX, PDF).
-

Final Project Topics (2/3)

- IoT Sensor Network: Observer for central node tracking sensor data, Facade to interface heterogeneous sensors, Strategy to process data streams (averaging, threshold).
- Vehicle Rental Service: Factory for vehicle objects (car, bike, van), Observer for rental availability notifications, Strategy for pricing (hourly vs daily).
- Online Education Platform: Builder for customizable course creation (modules, quizzes), Observer for live notifications (assignment graded), Strategy for recommendation algorithms (course suggestions).
- Chat/Messaging App: Observer for message listeners/receivers, Decorator to add features to messages (encryption, logging), Adapter for cross-protocol (XMPP vs REST API).
- Content Management System: Factory for different content types (article, video), Strategy for layout/rendering algorithms, Facade to encapsulate complex publishing workflows.
- Travel Booking Engine: Facade to integrate flights, hotels, car rentals; Strategy for best offer calculation; Observer for price drop alerts to users.
- Warehouse Automation: Builder for creating shipment orders, Observer for tracking package location, Bridge to decouple robot control (types of robots vs tasks).
- Photo Gallery App: Decorator to add image filters (grayscale, sepia) at runtime, Facade to simplify camera/capture APIs, Strategy for sorting (date, rating).
- Music Streaming Service: Facade for music library access, Observer for live track updates (lyrics display), Decorator for adding effects to audio streams.
- Banking System: Strategy for interest calculation (savings vs loan), Builder for assembling loan agreements, Observer for fraud detection alerts.
-

Final Project Topics (3/3)

- News Aggregator: Adapter to fetch news from different APIs (RSS, JSON), Observer to update UI feeds, Decorator to tag/categorize articles.
- Language Learning App: Strategy for spaced-repetition algorithms, Observer for progress tracking and reminders, Builder to assemble lesson modules.
- Fitness Tracker: Builder for custom workout plans, Observer for real-time health data (heart rate updates), Facade for handling different wearable device APIs.
- Ticket Reservation System: Factory for event/mode-specific tickets (concert, plane), Strategy for seating algorithms, Observer for notifying price or seat availability changes.
- Smart Agriculture: Observer for sensor alerts (moisture, weather), Facade to integrate irrigation/nutrient systems, Strategy for crop-growing schedules.
- Enterprise Resource Planning (ERP): Abstract Factory for different department modules (HR, Finance), Facade for unified data access, Observer for inventory reorder alerts.
- Blockchain Wallet: Strategy for transaction fee calculation, Observer for block/tx confirmations, Decorator to add multi-factor authentication flows.
- Augmented Reality App: Bridge for device abstraction (different AR SDKs), Adapter to integrate disparate sensors (camera, GPS), Strategy for rendering choices (stereoscopic vs pass-through).
- School Management System: Factory to create staff vs student profiles, Observer for grade/report notifications, Decorator to add roles or permissions.
- Retail Store Kiosk: Facade for combining payment and inventory systems, Strategy for price promotions, Observer to update sales dashboard in real-time.
-

Supplementary Resources

- Scholarly & Articles: A systematic literature review on design pattern selection (Naghdipour et al., Software: Practice and Experience, 2023) provides academic insight into applying patterns. Software design pattern selection approaches: A systematic review (example survey).
- Design Pattern Repositories: The java-design-patterns GitHub project contains many pattern examples in Java. Also, DovAmir/awesome-design-patterns is a curated list of pattern resourcesgithub.com.
- UML Tools: Popular free tools include PlantUML (text-based UML generator) and UMLet umltools.net. According to UML tools surveys, PlantUML, Modelio, UMLet, ArgoUML, etc., are widely used for drawing diagramsumltools.net. These help create class and sequence diagrams for homework.
- Video Tutorials: Refactoring.Guru's YouTube channel features visual design pattern guides. Derek Banas and Christopher Okhravi also have concise Java design pattern videos. The Clean Coders series by Uncle Bob (on SOLID) is useful as well.
- Java Repositories: Many open-source Java projects follow these patterns (e.g., Spring Framework uses Factory and Strategy). Browse Maven Central examples or the Head First Design Patterns sample code repo.
- Additional Reading: Clean Code by Robert C. Martin and Head First Design Patterns by Freeman et al. are course texts. Online blogs (Refactoring.Guru, Baeldung) and conference talks (e.g. Effective Java patterns) further reinforce concepts.